



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Харилцаа, синхрончлолын механизмууд

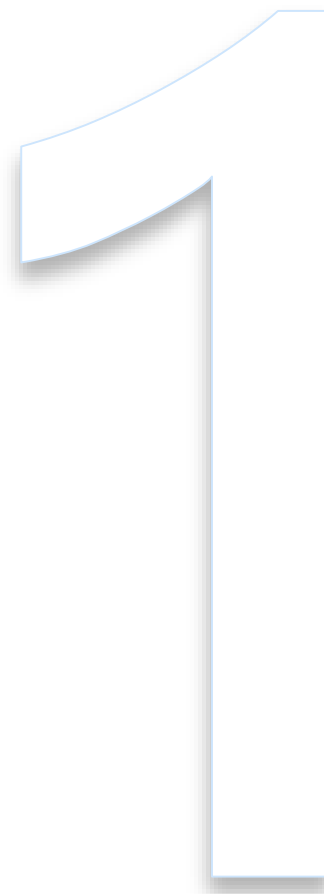
Лекц 13

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

Лекцийн агуулга

- Харилцах механизмууд
- Синхрончлолын механизмууд
- Дундын санах ой
- Мессеж солилцоо



- Зэрэгцээ байдлын төрлүүд
 - Физик зэрэгцээ байдал
 - Логик зэрэгцээ байдал
- Зэрэгцээ программчлалын төрлүүд
 - Параллел программчлал
 - Хамтарсан урсгалт (Multithreaded) программчлал
 - Хуваарилагдсан (Distributed) программчлал

- Зэрэгцээ ажиллагааны түвшин
 - Low-level зааврууд
 - *Дээд түвшний хэлний командууд*
 - *Программын дэд хэсгүүд*
 - Тусдаа программ бүтнээрээ
- Програмын дэд хэсгүүдийн зэрэгцээ байдал дээр бид анхаарна.
 - Дэд хэсгүүд хоорондын **харилцаа** болон **синхрончлол**-ыг удирдах
 - Үүний тулд дээд түвшний хэлний *ил бүтэцжүүлэлт* хэрэгтэй (Дээд түвшний хэлний командууд)

- Функцийн бодит биелэлтэд нөлөөлөх шаардлагууд
 - бие биентэйгээ харилцах,
 - Мэдээлэл солилцох (тооцооллын хэсэгчилсэн үр дүн, хяналтын харилцаа)
- Онолын хувьд хоёр суурь механизмд хуваадаг:
 - Харилцаж буй хоёр талыг авч үздэг (үйлдвэрлэдэг, хэрэглэдэг)
 - *Үйлдвэрлэгч ба хэрэглэгч*-ийн бодит харилцааны санаанд суурилдаг
- ***Дундын санах ойн харилцаа***
 - Оролцогчид, жнь, олон өөр процессууд утгыг ижил санах ойн зайд бичихээр/уншихаар хандах эрхтэй.
- ***Мессеж солилцоонд суурилсан харилцаа***
 - Send/Receive, Channel

- Процессууд дээрх идэвхжлийн харилцан эрэмбийг хянах
 - тооцооллын зөв байдлыг хангахад зайлшгүй шаардлагатай
- Дундын санах ойн харилцааны хувьд зайлшгүй бөгөөд синхрончлолын механизмууд ил аргаар явагддаг.
 - **харилцан хориглох (mutual exclusion)**: кодын эгзэгтэй тусгай мужуудад хэд хэдэн процесс нэгэн зэрэг нэвтрэх боломжгүй
 - **нөхцөлт синхрончлол (condition synchronization)**: тохиромжтой нөхцөл үүсэх хүртэл процессын гүйцэтгэлийг түр зогсоох.
- Мессеж солилцооны суурилсан харилцааны ерөнхийдөө далд явагддаг.
 - **Asynchronous**: send үйлдэл нь блоклогдохгүй, receive блоклогддог
 - **Synchronous**: send, receive блоклогддог

Дундын санах

- Харилцаа зөв байхын тулд *уралдааны нөхцөл* болон бусад буруу нөхцлөөс байдлаас зайлсхийх.
 - *Харилцан хориглох*. Зэрэгцсэн процессуудад **эгзэгтэй муж (critical section)** буюу дундын нөөцөд ханддаг кодын хэсгийг хамтран биелүүлэхийг зөвшөөрдөг: биелэлтийн хувьд кодын хэсгийг бүтнээр нь **атомик** гэж үзнэ.
 - *Нөхцөлт синхрончлол*. Тусгай утга авдаг хувьсагч зэрэг тохиромжтой нөхцлийг үүсэх хүртэл процессын биелэлтийг түр зогсоох.
- Эдгээр хоёр төрлийн синхрончлолыг хоёр техник ашиглан хийдэг:
 - **завгүй хүлээх** (*busy waiting*, мөн *spinning* гэж нэрлэдэг)
 - **блок** (мөн *төлөвлөгч суурьт синхрончлол* гэж нэрлэдэг).

- Эгзэгтэй мужид орох, эсвэл өгөгдсөн нөхцөл биелсэн процессууд
 - Нөхцөлийг үнэн болох хүртэл нь шалгах тасралтгүй циклийг гүйцэтгэдэг
- Нэг төвлөрсөн процессортой машин дээр тохиромжгүй
 - Хүлээлтийн үеэр, CPU идэвхтэй ажилласаар байдаг,
 - Өөр процессоос гарах нөхцлийг хүлээж CPU-г сул зогсоох нь ашиггүй.
- Олон процессортой машин дээр ашигладаг бол үр дүнтэй
 - HW түвшинд, жнь, дата шугам болон дотоод сүлжээн дэх өгөгдөл дамжуулалтыг синхрончлох
- Синхрончлолын олон механизмд идэвхтэй хүлээлтийг ашигладаг:
 - **түгжээ** (*lock*, харилцан хориглолтыг хэрэгжүүлэх)
 - **саад** (*barrier*, нөхцөлт синхрончлолыг хэрэгжүүлэх)


```
process Pi {  
  non-critical section;  
  acquire_lock( B );  
  critical section;  
  release_lock( B );  
  non-critical section;  
}
```

- HW дэмждэг, маш тодорхой атомик үйлдэл
 - test_and_set(B)
- Анхны утга:
 - **bool B = false;**

```
void acquire_lock( ref B: bool ) {  
  while test_and_set( B ) do { skip; }  
}  
  
void release_lock( ref B: bool ) {  
  B = false;  
}
```

- Параллел програмчлалд хэрэглэгддэг нөхцөлт синхрончлолын механизм.
 - Ихэвчлэн давталт ашиглан эцсийн шийдийг гаргах n процессоор хэрэгждэг алгоритм (жнь, матрицын манипуляци)
 - Давталт бүрийн төгсгөлд процесс бүр дараагийн давталт руу шилжихээс өмнө бусад бүх процессийн ажиллагаа дуусахыг хүлээнэ.
- $i = 1 \dots n$ байх P_i процесс бүр дараах загвартай байна

```
process Pi {  
  while B true do {  
    code of task i;  
    wait for all other Pj to terminate;  
    // barrier  
  }  
}
```

- 0-ээр эхэлдэг глобал тоолуурыг ашиглаж болно. Процесс бүр дуусахдаа ++
- Тоолуурт хандах үйлдлүүд атомик байх ёстой.

- Өмнөх хэсэгт үзсэн идэвхтэй хүлээх механизм нь хэд хэдэн сул талтай.
 - Процессороос процесс олон. Процессууд бүгд CPU-г ашиглахыг хүлээж байхад нэг процессийг идэвхтэй хүлээхэд CPU-г ашиглах нь тохиромжгүй
 - Тусгай өгөгдлийн төрөл, хэл шинжлэлийн бүтэцлэлт байхгүй. Энгийн хувьсагч ашигладаг. Программчлалын алдаа гаргах магадлал өндөр.
- **Төлөвлөгч суурьт (block-based) синхрончлол:**
 - Процессын хүлээж буй эвентийг тухайн процесстой холбодог. Эвент тохиолдоход, төлөвлөгч тухайн эвенттэй холбогдсон процессууд биелэгдэхэд бэлэн байгааг мэддэг.
 - Тусгай утга авдаг хувьсагчийг хэрэглэдэг, утгууд дээр нь синхрончлолд зориулсан тусгай үйлдлүүдийг дэмждэг. Онолын түвшинд өгөгдлийн төрөл гэж болно, гэхдээ тэдгээр нь дээд түвшний хэл дээр бүтэц хэлбэрээр ховорхон харагддаг.

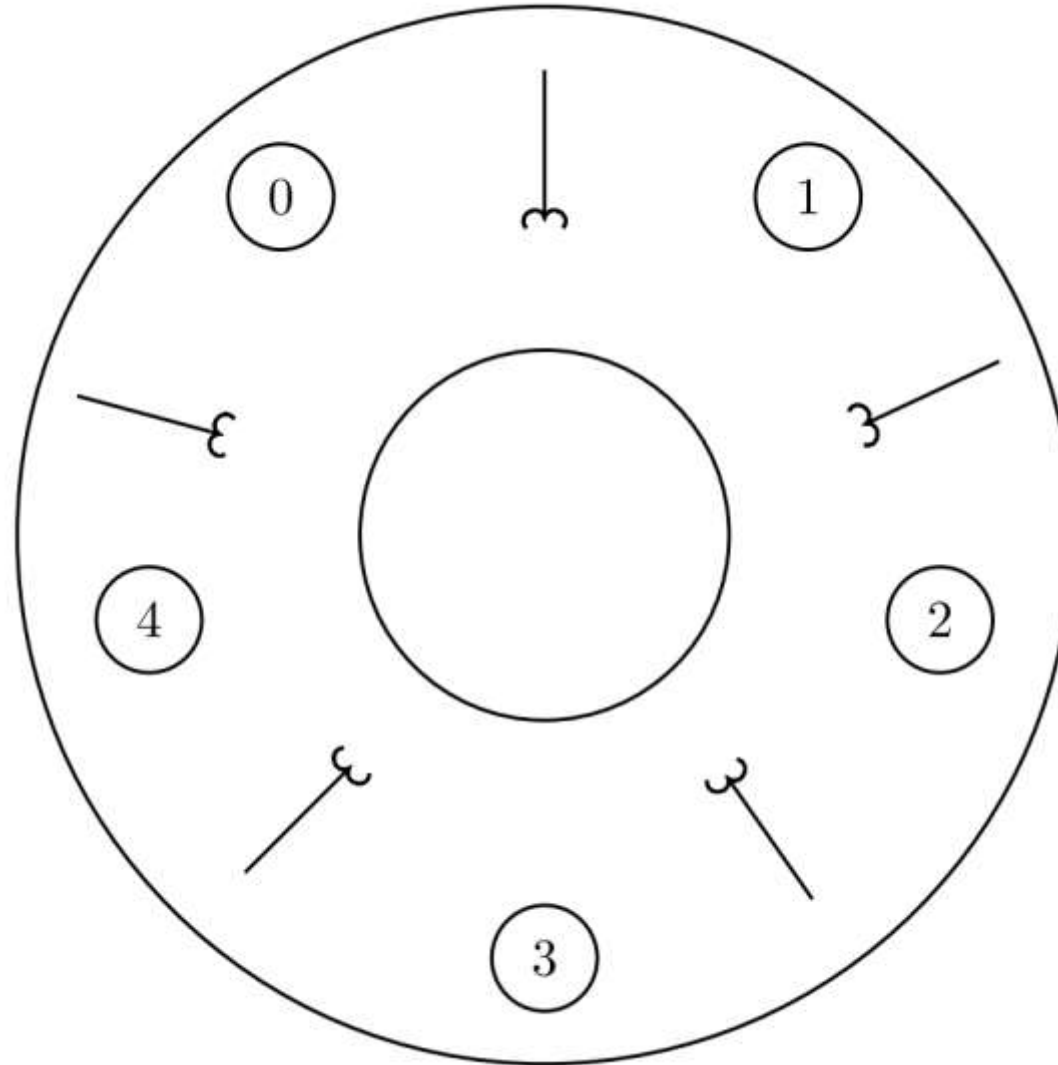
- Dijkstra, 1960s, дунын C/O-н синхрончлолын анхын хэрэгсэл, Алгол 68. Бараг бүх сан, хэлэнд хэрэглэгдэж байна
 - "мөргөлдөөнөөс" зайлсхийх процесс "трафик" зохицуулалтын бүтэц
 - Төмөр замын хөдөлгөөнийг зохицуулдаг бодит төхөөрөмжтэй төстэй
 - Утга: Семафорын өнгө, Замын хэсэг чөлөөтэй эсэхийг илтгэн галт тэрэг нэвтрэхийг зөвшөөрдөг/хориглодог.
 - Үйлдэл: Галт тэрэг замын хэсгийг эзлэх (ногооноос улаан) эсвэл суллах (улаанаас ногоон) үед семафорын өнгийг тусгай үйлдлээр удирддаг.
- Семафор өгөгдлийн бүтэц
 - 0-ээс их бодит тоон утгын олонлог
 - P, V гэсэн хоёр атомик үйлдэлтэй

```
sem s
```

```
P ( s )
```

```
V ( s )
```

First-In-First-Out (FIFO)



The dining
philosophers
problem

ДУНДЫН С/О ТӨЛӨВЛӨГЧ СУУРТ СИНХРОНЧЛОЛ Семафор жишээ

```
sem[] forks[5];
for ( i = 0, i < 5, i += 1 ) { forks[ i ] = 1; }

process Philosopher( i ){
    sem first_fork, second_fork;
    if( i == 0 ){
        first_fork  = forks[ 4 ];    // the one on the right
        second_fork = forks[ 0 ];    // the one on the left
    } else {
        first_fork  = forks[ i ];    // the one on the left
        second_fork = forks[ i-1 ];  // the one on the right
    }
    while true {
        think;
        P( first_fork );    // acquire first
        P( second_fork );   // acquire second
        eat;
        V( first_fork );    // release first
        V( second_fork );   // release second
    }
}
```

Philosopher(0) || Philosopher(1) || Philosopher(2) || Philosopher(3) || Philosopher(4)

- Өгөгдлийн хийсвэрлэл, бүтэц талаасаа семафор хангалттай байдаггүй
 - Семафор нь процесс дундын хувьсагч, P ба V-г ашиглан энэ хувьсагчийг зөв ашиглах, улмаар зөв байх бүх эрсдэлийг програмист хариуцна
 - Семафор нь харилцан хориглох, нөхцөлт синхрончлол хоёрт ажиллах боломжтой. Хоёр ойлголтын ялгаатай байдлыг далдлах хэрэгтэй
- Анх Dijkstra, 1970s. Бринч Хансен, К.А.Р. Hoare хөгжүүлэн, Java гм
- **Монитор** нь хийсвэр өгөгдлийн төрөл
 - Хийсвэр объектын дүрслэлийг төлөвтэй хамт инкапсуляц хийсэн, тухайн объект дээр ажиллах зөвшөөрөгдсөн үйлдлүүдтэй.
 - Мөн мониторуыг үүсгэх үед ашигладаг эхлүүлэх командуудтай.

```
monitor MonitorName {  
    variables;  
    initialisation commands;  
    procedures;  
}
```


- Монитор байгаа бол байнгын хувьсагч байдаг
 - Нөхцөлт хувьсагч - нөхцөлт синхрончлолыг зөвшөөрдөг
- Мониторын гадна талд зөвхөн процедурын нэрс л харагдана
 - Процессууд байнгын хувьсагч дээр ажиллахад ашиглаж болох зөвшөөрөгдсөн үйлдлүүд.
- Монитор ашигласан идэвхгүй, идэвхтэй хэсгүүдийг ялгаж чаддаг.
 - Процесс дундын өгөгдлийн бүтцийг агуулдаг.
 - Мониторуудын процедурыг ашиглан харилцан ажилладаг.
- Программист нь мониторын гадаад интерфэйсийг зөвхөн мэддэг байх
 - Дараалсан программчлалаас ялгаатай нь мониторуыг өөр өөр цагт/хурдтай нэгэн зэрэг идэвхтэй ажилладаг зэрэгцээ процессууд дундаа хэрэглэдэг.
- Харилцан хориглох нь монитор бүтэц программистаас юу ч хийх шаардаггүй

Мессеж солилцоо

- Хуваарилагдсан архитектурын хувьд түгээмэл харилцааны загвар
 - Нэрлэх механизм: нэр, порт, суваг (channels)
 - Асинхрон харилцаа
 - Синхрон харилцаа
 - Алсын процедур дуудалт

- Анх програмчлалын хэлний түвшинд Оссат-д, янз бүрийн онолын загварууд (жишээ нь, π -calculus), хэлүүдэд.
- Онолын хувьд хоёр оролцогчийг холбох боломжийг олгодог аливаа харилцааны бүтцийн хийсвэрлэл юм
 - дундын хувьсагч, монитор зэргийг сувгийн хэрэгжилт гэж үзэж болно.

```
channel chanName( Type )  
send chanName( data )  
receive chanName( var )
```

- Нэр томьёоны хувьд физик сувгууд, бодит сүлжээний хийсвэр загварт хэрэглэдэг
 - One-way: өгөгдлийн урсгал нэг чиглэлтэй, link
 - Two – way: өгөгдлийн урсгал нь хоёр тийшээ
 - Many-to-many: Mailbox

- Асинхрон харилцаанд илгээгч нь мессеж илгээсний дараа хүлээн авагчийг хүлээн авах үйлдлийг гүйцэтгэхийг хүлээхгүйгээр тооцоогоо үргэлжлүүлнэ.
 - Суваг дээр дундын, хязгааргүй урттай мессеж FIFO дараалалтай. Атомик хандана.
 - `send chanName( data )` үйлдэл блоклогдохгүй. Хүлээн авагчийн хариуг хүлээхгүйгээр тооцооллоо үргэлжлүүлнэ.
 - `receive chanName( var )` үйлдэл блоклогдоно. Сувгаас эхний мессежийг өөртөө аваад усгана. Суваг хоосон бол хүлээнэ.

```
channel request( int client, char data );  
channel response1( char res );  
channel response2( char res );
```

```
process Client1{  
  char values;  
  char results;  
  ... // Definition of values  
  send request( 1, values );  
  receive response1( results );  
  ... // Usage of results  
}  
  
process Client2{  
  char values;  
  char results;  
  ... // Definition of values  
  send request( 2, values );  
  receive response2( results );  
  ... // Usage of results  
}
```

```
process Server{  
  int client;  
  char values;  
  char results;  
  ... // Initialisation  
  while true {  
    receive request( client, values );  
    if client = 1 then {  
      ... // Processing of values  
      send response1( results );  
    }  
    if cliente = 2 then {  
      ... // Processing of values  
      send response2( results );  
    }  
  }  
}
```

- `send`, `receive` үйлдлийг харилцаа бодитоор хэрэгжих үед процессуудыг синхрончлох боломжийг олгодог гэж харж болно.
 - Мессеж дараалал хамгийн ихдээ 1 мессеж. Бодит байдалд үйлдлүүд өөр цаг үед явагддаг, илгээгч нь `send` явсны дараа мессежийг хүлээн авсныг баталгаажуулах дохиог хүлээнэ.
 - Алдаатай ажиллах. Яг алдаа гарсан бол шууд алдаагаа засах ажлаа хийнэ
 - `send chanName( data )` болон үйлдэл блоклогдоно. Илгээгч хүлээн авагчийг хариу өгөх хүртэл хариу хүлээнэ. Дор хаяж нэг процесс блоклогддог (ядахдаа `delay`) учир зэрэгцээ байдал буурдаг.
- Бас нэг сул тал нь программистад илүү төвөгтэй.
 - Түгжрэл (`deadlock`) болон бусад алдааг гаргах нь илүү дөхөм
 - 2 процесс `send` болон `receive` хоёуланг биелүүлэх боломжтой, эхлээд хоёул `send` хийвэл түгжирнэ

- Олон хэлээр (Java-г оруулаад) ашигладаг Remote Procedure Call
 - Практикт, ялангуяа клиент-сервер харилцан үйлчлэлд өргөн хэрэглэгддэг
 - Синхрон send-ээс ялгаатай нь илгээгч мессеж илгээсний дараа тооцооллыг үргэлжлүүлэхийн өмнө хүлээн авагчаас хариу мессеж ирэхийг (мессеж хүлээн авсан гэсэн мэдэгдэл биш) хүлээдэг.
 - RPC нь алсаас биелүүлдэг процедурыг ихэвчлэн өөр машин дээр ажилладаг
- RPC-ийн хувьд хүлээн авагч тал нь тухайн дуудалтыг хүлээж байдаг идэвхтэй процесс байхгүй.
 - Хүлээн авагч мессежийг хүлээн авахдаа түүнийг боловсруулах (параметруудийг дамжуулах, процедурыг гүйцэтгэх, үр дүнг дуудагч руу дамжуулах) шинэ процессыг бий болгодог.
 - Мессеж солилцооны хувьд RPC нь дуудалт гэж нэрлэгддэг зөвхөн илгээх ил үйлдэл гүйцэтгэдэг боловч хүлээн авах ил үйлдэл хийдэггүй.

```
module modName{  
  op proc1( type of formal parameters ) returns ( type of result )  
    // headers  
  ...  
  body {  
    local variables;  
    initialisation code;  
    proc proc1( formal parameters ) returns ( result ){  
      procedure body  
    }      // implementation  
    ...  
    local procedures;  
    local processes;  
  }  
}
```

```
call modName.proc1( actual parameters )
```



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА